

Vehicle management, explained simply

This note explains the vehicle part of the system: the Java classes that do the work, the three JSP pages the user sees, and the design patterns you can name in the viva.

1. The whole idea in one minute

Vehicle management lets a customer **find a car** and lets fleet staff **add, edit, move, retire, or delete** a car.

A vehicle is one row in the `vehicles` table. It has a registration number, brand, model, seats, gearbox, fuel, price per day, deposit, a home branch, a current location, and a status. The status is one of `AVAILABLE`, `UNAVAILABLE`, `MAINTENANCE`, or `RETIRED`.

The browser never talks to the database. The path is always:

JSP page → VehicleController → VehicleService → VehicleRepository → MySQL

The JSP only displays data and sends a form. The controller reads the HTTP request. The service applies the business rules. The repository runs the query.

Search. The customer opens Find a ride. The form sends GET `/vehicles` with optional filters. The service returns cars that match. Retired cars are never shown.

Date search. If both pickup and return dates are filled, the service also hides cars that are under maintenance, unavailable, retired, or already booked for those dates. A booking blocks the car only when its status is `PENDING`, `APPROVED`, or `ONGOING`.

Add. Staff open `/vehicles/new`, fill the form, and POST `/vehicles`. The plate is stored in uppercase. A duplicate plate is rejected. If location is blank, it becomes the branch name.

Edit. Staff POST `/vehicles/{id}`. The existing row is updated. Keeping the same plate is allowed. Using another car's plate is not.

Transfer. Staff move the car to another branch. The system writes a log row and updates the car's branch and location. A retired car cannot be moved. A car cannot be moved to the branch it is already at.

Retire. Status becomes `RETIRED`. The row stays in the database, but public search no longer lists it.

Delete. The car and its bookings, payments, maintenance, and transfer log are removed. Children are deleted before the car, because of foreign keys.

2. Who is allowed to do what

Rules live in `SecurityConfig.java`. Hiding a button in the JSP is not enough. Spring Security blocks the URL.

Request	Who
GET <code>/vehicles</code> and GET <code>/vehicles/{id}</code>	Everyone, including a guest
GET <code>/vehicles/new</code> , GET <code>/vehicles/{id}/edit</code>	ADMIN, FLEET_COORDINATOR, OPERATIONS_MANAGER
POST create, update, transfer, retire, delete	Those same three roles
Maintenance pages	ADMIN and FLEET_COORDINATOR only

Every staff form also sends a hidden CSRF token. Spring rejects the POST if that token is missing.

3. Backend, file by file

3.1 VehicleController.java — reads the web request

File: `src/main/java/com/lankaride/vehicle/VehicleController.java`

This class does not contain business rules. It translates HTTP into method calls and chooses which JSP to show.

- `@Controller` tells Spring this class handles web pages and returns a JSP name, not JSON.
- `@RequestMapping("/vehicles")` means every method below starts with `/vehicles`.
- The constructor receives `VehicleService` and `FleetDefaultService`. Spring creates those objects and passes them in. This is dependency injection. The controller never writes `new VehicleService()`.
- `savedModels()` and `savedCategories()` are marked `@ModelAttribute`. Before a vehicle page renders, Spring calls them and puts the fleet-default lists on the page. The form uses those lists for the “saved model” dropdown and the category suggestions. They are not separate URLs.

list — GET `/vehicles`

- Each filter is `@RequestParam(required = false)`. A missing filter is null, so “show everything” still works.
- `category` is text. `gearbox`, `fuelType`, and `status` are enums. Spring converts the form text `AUTOMATIC` into `GearboxType.AUTOMATIC`.
- `branchId` is a number. Prices are `BigDecimal`. Dates are `LocalDate`.
- `availableOnly` defaults to false when the checkbox is absent.
- If both dates exist and return is before pickup, the method stores an error and ignores the dates.
- If pickup is before today, it stores “Pickup date cannot be in the past” and ignores the date search.

- It then calls `vehicleService.search(...)`. The last argument is `availableOnly || dateSearch`. A date search always asks for cars that are free on those dates.
- It also puts branches, gearbox values, fuel values, and the original filter values back on the `Model`, so the form can show what the user just typed.
- `return "vehicle/list"` means "render `WEB-INF/views/vehicle/list.jsp`". The method does not build HTML.

detail — GET /vehicles/{id}

- `@PathVariable Long id` is the number in the URL, such as `/vehicles/1`.
- Optional `pickupDate`, `returnDate`, and `branchId` are kept so the Book button can pass the customer's search dates into the booking form.
- The page receives the vehicle, its transfer history, and all branches (for the transfer dropdown).
- It returns `"vehicle/detail"`.

createForm — GET /vehicles/new

- Puts a brand-new empty `Vehicle` on the model.
- Loads branches and the three enums for the dropdowns.
- Sets `formAction` to `/vehicles`. The same JSP is used for add and edit. This value tells the form where to POST.

create — POST /vehicles

- `@Valid @ModelAttribute("vehicle")` fills a `Vehicle` from the form fields and runs the validation annotations on the class.
- `BindingResult` must be the next argument. If it has errors, the method shows the form again. It does not redirect, so the user's text stays on the page.
- `branchId` is a separate request parameter, because the branch is a related object, not a plain text field.
- On success, `vehicleService.create` saves the car. A flash message is stored, and the browser is redirected to `/vehicles/{newId}`.
- A flash message lives for one redirect only. That is why "Vehicle created" appears on the detail page after save.
- If the service throws `IllegalArgumentException` (duplicate plate, bad branch, bad text), the message is shown on the form.

editForm and update

- GET `/vehicles/{id}/edit` loads the existing car and sets `formAction` to `/vehicles/{id}`.
- POST `/vehicles/{id}` validates, then calls `vehicleService.update`. Success redirects to the detail page with "Vehicle updated".

transfer, retire, delete

- Transfer reads `toBranchId`, an optional note, and `auth.getName()` (the logged-in username). Success and failure both redirect back to the detail page. Success uses `message`. Failure uses

`error`.

- Retire calls `vehicleService.retire` and redirects to the vehicle list.
- Delete reads the registration number first, so the message can say which car was removed, then calls `deletePermanently`, then redirects to the list.

`populateForm` is a private helper. Create and update both need the same dropdowns, so that list is written once.

3.2 VehicleService.java — the rules

File: `src/main/java/com/lankaride/vehicle/VehicleService.java`

- `@Service` marks this as the business-logic bean.
- `BLOCKING_STATUSES` is PENDING, APPROVED, and ONGOING. Those bookings occupy the car. DENIED, COMPLETED, and CANCELLED do not.
- The constructor receives six repositories: vehicles, branches, bookings, transfers, maintenance, and payments. Delete and search need more than the vehicle table.

search

- There are two `search` methods. The short one calls the long one with no dates and `availableOnly = false`.
- If `availableOnly` is true and the user did not pick a status, the service forces status `AVAILABLE`.
- `vehicleRepository.search` runs the database filter: category, gearbox, fuel, branch, min price, max price, status. Retired cars are already removed inside that query.
- If both dates are valid, a second Java filter runs on that list. A car is kept only when it is not MAINTENANCE, UNAVAILABLE, or RETIRED, and `bookingRepository.countOverlapping` returns 0.
- Overlap means: same vehicle, status in the blocking list, and the booking's pickup is on or before the requested return, and the booking's return is on or after the requested pickup. That is the standard "ranges overlap" test.
- If there are no dates but `availableOnly` is true, the service keeps only `AVAILABLE` cars.

getById loads one car. If the id does not exist, it throws "Vehicle not found".

create is `@Transactional`, so the checks and the insert commit together or not at all.

- `normalize` trims text, uppercases the plate, and checks seats and money.
- `existsByRegistrationNumber` stops a duplicate plate.
- The branch must exist.
- A blank location becomes the branch name.
- `save` inserts the row and returns it with the new id.

update loads the old row and copies fields onto it. It does not insert a second car.

- The duplicate-plate check uses `filter(v -> !v.getId().equals(id))`. The car may keep its own plate. It may not take another car's plate.
- Every editable field is copied, including status and branch, then `save` updates the same row.

transfer

- A RETIRED car throws "Retired vehicles cannot be transferred".
- The destination branch must exist.
- If the destination id equals the current branch id, it throws "already at ...".
- A new `BranchTransfer` stores the vehicle, the old branch, the new branch, the old location, the staff username, the note, and the current time.
- The vehicle's branch and current location are then changed to the new branch.
- Both saves are one transaction. If the log save failed, the move would roll back too.

retire sets status to RETIRED and saves. The public list query hides that status, so the car leaves the catalogue without being deleted.

deletePermanently deletes in this order, then flushes so the database really removes each group before the next:

1. Payments of every booking of this vehicle.
2. Those bookings.
3. Maintenance records of this vehicle.
4. Branch-transfer rows of this vehicle.
5. The vehicle row.

The order matters. A booking points at the vehicle, and a payment points at the booking. Deleting the vehicle first would break the foreign keys.

normalize

- Registration: required, 3–20 characters, then uppercased.
- Category, brand, and model: required labels, letters, numbers, spaces, and hyphens.
- Seats: 1 to 20.
- Price and deposit: required money, greater than zero, within the shared cap.
- Features, location, and photo URL: optional, with length and URL checks.
- Gearbox, fuel, and status must not be null.

`InputChecks` is a small shared helper in `com.lankaride.common`. Vehicle, booking, and other forms use the same text rules instead of copying them.

3.3 VehicleRepository.java — the query

File: `src/main/java/com/lankaride/vehicle/VehicleRepository.java`

- This is an interface. There is no class body. It extends `JpaRepository<Vehicle, Long>`, so Spring already provides `save`, `findById`, `findAll`, `delete`, and `flush`.
- `findByRegistrationNumber` and `existsByRegistrationNumber` are understood from the method name. Spring writes the SQL.
- `countByStatus` counts cars in one status. The dashboard uses this style of method.
- `search` is a JPQL query, written in the `@Query` string. It selects `Vehicle` objects, not raw table rows.

- `v.status <> RETIRED` always removes retired cars.
- Each filter looks like `:category IS NULL OR :category = '' OR LOWER(v.category) = LOWER(:category)`. A blank filter matches every car. Category compare ignores capital letters.
- `v.branch.id = :branchId` joins the branch. `pricePerDay` is compared with min and max.
- Results are ordered by brand, then model.
- This query does not know about bookings. Date overlap is done later in the service.

3.4 Vehicle.java — the table row as a Java object

File: `src/main/java/com/lankaride/vehicle/Vehicle.java`

- `@Entity` and `@Table(name = "vehicles")` map this class to the MySQL table.
- `id` uses `GenerationType.IDENTITY`. MySQL assigns the number on insert.
- `registrationNumber` is required, unique, 3–20 letters, numbers, or hyphens.
- `category`, `brand`, and `model` are required text with the same character rule.
- `seats` is an int from 1 to 20.
- `gearbox` and `fuelType` are enums stored as text (`EnumType.STRING`), so the column contains `AUTOMATIC`, not a magic number. Defaults are `AUTOMATIC` and `PETROL`.
- `features` and `photoUrl` are optional. A photo must be empty or start with `http://` or `https://`.
- `pricePerDay` and `depositAmount` are `BigDecimal`, not `double`, so money does not pick up binary rounding. The column allows 10 digits with 2 decimal places, and the value must be greater than zero.
- `@ManyToOne` on `branch` means many vehicles belong to one branch. The foreign key column is `branch_id`. Fetch is `EAGER`, so loading a vehicle also loads its branch name.
- `currentLocation` is a short text description of where the car is now.
- `status` defaults to `AVAILABLE`.
- The rest of the file is get and set methods. JSP expressions such as `${vehicle.brand}` call `getBrand()`. Spring form tags such as `path="brand"` call `getBrand` and `setBrand`.

3.5 The small types around a vehicle

- `VehicleStatus.java` is the enum `AVAILABLE`, `UNAVAILABLE`, `MAINTENANCE`, `RETIRED`.
- `Branch.java` is the depot: id, unique name, address. Table `branches`.
- `BranchTransfer.java` is one move: vehicle, from branch, to branch, old location, time, staff username, and note. Table `branch_transfers`.
- `BranchRepository` and `BranchTransferRepository` are the same Spring Data style. Transfers are listed with `findByVehicleIdOrderByTransferredAtDesc`, newest first.

4. The three JSP pages

JSPs live in `src/main/webapp/WEB-INF/views/vehicle/`. They cannot be opened as files in the browser. Spring forwards to them after the controller runs. `${...}` prints a value. `<c:if>` and

`<c:forEach>` are JSTL tags. `<sec:authorize>` shows or hides HTML by role. It does not replace the security rules in `SecurityConfig`.

4.1 list.jsp — Find a ride

- Sets the browser title to “Find a ride” and includes the shared header.
- The heading and the sentence under it change when dates were searched. If `pickupDate` and `returnDate` are present, it says “Available {from} → {to}”. Otherwise it says “Browse the fleet...”.
- The Add vehicle button is wrapped in `sec:authorize` for ADMIN, FLEET_COORDINATOR, and OPERATIONS_MANAGER. A customer does not see it. A customer who types the URL still gets blocked by Spring Security.
- The filter form is `method="get"` and `action="/vehicles"`. GET is correct because search does not change data and the URL can be bookmarked.
- Field names match the controller exactly: `branchId`, `pickupDate`, `returnDate`, `category`, `gearbox`, `fuelType`, `minPrice`, `maxPrice`, `availableOnly`.
- The branch, gearbox, and fuel dropdowns loop over `${branches}`, `${gearboxes}`, and `${fuelTypes}` that the controller placed on the model. The selected option is marked when it equals the value just searched.
- The checkbox sends `availableOnly=true` only when ticked. When it is unticked, the parameter is absent and the controller’s default false is used.
- Search submits the form. Reset is a link back to `/vehicles` with no query string.
- `<c:forEach items="${vehicles}">` prints one card per car. The card link is `/vehicles/{id}` and, when the user searched with dates or a branch, those values are added as query parameters. That is how the detail page later builds the booking link with the same dates.
- Each card shows the photo, or a fallback icon if `photoUrl` is empty or the image fails to load.
- The card text uses `v.brand`, `v.model`, `v.status`, registration, branch name, category, seats, gearbox, fuel, and price per day.
- If `vehicles` is empty, the page says no cars match the filters.

4.2 detail.jsp — one car

- The back link returns to `/vehicles`.
- The photo block shows `vehicle.photoUrl`, or the “No photo” fallback.
- The heading is brand and model. The line under it is registration and branch name. The status pill uses the status name in lowercase as a CSS class.
- The spec grid prints category, seats, gearbox, fuel, features, location, price per day, and deposit. An empty features field prints a dash.
- If the visitor is logged in and the car is AVAILABLE, **Book this vehicle** points at `/bookings/new` and adds `vehicleId`, plus pickup date, return date, and branch when those were passed in. If no branch was in the search, it uses the car’s own branch id.
- A guest who is looking at an AVAILABLE car sees **Login to book** and **Register** instead.
- ADMIN and FLEET_COORDINATOR see **Add maintenance**, linking to `/maintenance/new?vehicleId=...`.

- ADMIN, FLEET_COORDINATOR, and OPERATIONS_MANAGER see Edit, Retire, and Delete. Edit is a GET link. Retire and Delete are POST forms. Each form includes the CSRF hidden input and a `confirm(...)` so a mis-click does not destroy data. The confirm dialog is only a browser warning. The server still checks the role.
- The transfer panel is shown to those same three roles. The form POSTs to `/vehicles/{id}/transfer`. The current branch's option is `disabled`, so it cannot be chosen. Note is optional.
- The transfer log loops over `${transfers}` and prints time, from branch, to branch, the username, and the note. If the list is empty, it says no transfers were recorded.

4.3 form.jsp — add and edit share one page

- The title is "Add vehicle" when `vehicle.id` is null, and "Edit vehicle" when an id exists.
- The form POSTs to `${formAction}`. For a new car that is `/vehicles`. For an edit it is `/vehicles/{id}`. The controller set that value.
- `<form:form modelAttribute="vehicle">` binds the form to the `Vehicle` object. `path="registrationNumber"` reads and writes `getRegistrationNumber` / `setRegistrationNumber`. The same pattern is used for category, brand, model, seats, gearbox, fuel, features, photo URL, price, deposit, location, and status.
- The CSRF hidden field is added by hand with `${_csrf.parameterName}` and `${_csrf.token}`.
- If saved models exist, a dropdown lists them. This dropdown's name is `savedModel` and it is **not** posted as a vehicle field. It only helps the browser fill the other boxes.
- The script at the bottom listens for a change on that dropdown. The chosen `<option>` carries `data-brand`, `data-model`, `data-category`, `data-seats`, `data-gearbox`, and `data-fuel`. The script copies those into the real inputs. Nothing is saved until the user presses Save. The server validates the posted values again.
- Category has a `<datalist>` of saved category names. The user can still type a new category.
- Gearbox, fuel, and status are `<form:select>` tags filled from the enum arrays.
- Home branch is a normal `<select name="branchId">`, not a `form:select` path, because the controller reads `branchId` as its own parameter and then loads the `Branch`.
- On edit, the option whose id equals `vehicle.branch.id` is marked selected.
- `<form:errors path="...">` prints the message from `@NotBlank`, `@Min`, and the other annotations when validation fails.
- Save submits the form. Cancel links back to `/vehicles` and discards the typed changes.

5. Design patterns in this code

Name a pattern only if you can point at a class. These are the ones that are really here.

Model-View-Controller

`Vehicle` is the model. `list.jsp`, `detail.jsp`, and `form.jsp` are the views. `VehicleController` is the controller. The controller chooses the view. The view does not run the duplicate-plate rule. The

model does not know about HTTP.

Front Controller

The application does not have one servlet per page. Spring's `DispatcherServlet` receives every request and sends `/vehicles` to `VehicleController`. That single entry point is the Front Controller.

Layered architecture

Four layers, top to bottom: JSP, controller, service, repository. Each layer calls only the one below it. The JSP does not call `VehicleRepository`.

Dependency injection (Inversion of Control)

`VehicleController` and `VehicleService` ask for their helpers in the constructor. The Spring container creates the objects and wires them. The classes do not construct their own repositories. By default each of these beans is one shared instance for the whole application. Request data stays in method arguments and in the database, not in fields that change per user.

Repository

`VehicleRepository` hides the query. The service says `search` and `save`. It does not write JDBC. Spring Data supplies the implementation of the interface.

Facade

`VehicleService` is the one door for vehicle use cases. Deleting a car needs payments, bookings, maintenance, and transfers. The controller calls `deletePermanently` once and does not know that order. Search also hides two steps: the SQL filter and the booking-overlap filter.

Template Method

`JpaRepository` already implements the fixed steps for save, find-by-id, and delete. `VehicleRepository` only adds the extra `search` query. The framework algorithm stays the same. Your query is the varying step.

Data Mapper

JPA and Hibernate map the `Vehicle` object to the `vehicles` table using `@Entity`, `@Column`, and `@JoinColumn`. The Java class does not contain the INSERT statement.

Post-Redirect-Get

After create, update, transfer, retire, and delete, the controller sends a redirect and a flash message. Refreshing the next page does not submit the form a second time.

Say this if they ask for a pattern you did not use. There is no Strategy class for each filter, no Factory that builds vehicles, and no State class for each status. `VehicleStatus` is an enum. The rules are ordinary `if` checks inside `VehicleService`.

6. A short answer you can speak

Vehicle management is the fleet catalogue. Guests can search and open a car. Fleet staff can add, edit, transfer, retire, and permanently delete.

The page is a JSP. `VehicleController` reads the request and picks the page. `VehicleService` checks the plate, the dates, the branch move, and the delete order. `VehicleRepository` runs one JPQL search that ignores empty filters and always hides retired cars. If the customer also sent dates, the service removes cars that overlap a pending, approved, or ongoing booking.

The structure is MVC. Spring injects the service into the controller and the repositories into the service. The service is a facade over several tables. Spring Data's repository is a template for the common database operations.